SOFTWARE-IN-THE-LOOP RECONSTRUCTION FOR VERIFIED TERMINAL-AGENT TRAINING

Anonymous authors
Paper under double-blind review

ABSTRACT

Existing training datasets for terminal agents concentrate on software engineering, leaving scientific and other specialized domains comparatively underrepresented. Constructing tasks in these domains requires both reference solutions and verifiers that distinguish correct behavior from superficially valid outputs. Developing these components separately for each task limits the expansion of training data. We introduce *software-in-the-loop reconstruction*, which addresses both requirements by reusing executable software workflows. Each workflow serves as a reference solution and generates input–output examples under different configurations. An agent receives selected examples and reconstructs an editable program, whose outputs are compared with the workflow's outputs on hidden inputs. The same workflow therefore supports multiple tasks and supplies their verification targets, reducing the need to author solutions and expected outputs separately for each task. A hierarchical verifier combines domain-specific semantic comparisons with structural and predefined anti-shortcut checks, while public feedback supports iterative repair. We instantiate this method as SWR3000, comprising 3,000 tasks derived from 500 workflows across 46 software families and six domains. Across three attempts per task, Qwen3.8-Max solves 838 tasks and generates 1,422 verified trajectories, which we oversample to obtain 3,000 reconstruction-only training examples. Supervised fine-tuning of Qwen3.8-27B improves mean Terminal-Bench 2 performance from 47.94% to 53.56% across three seeds and achieves the highest mean performance among four matched-token corpus controls on all four reported evaluations. These results support workflow reconstruction as a means of expanding terminal-agent training across domains by reusing executable reference solutions and their outputs for verification. Code and evaluation data are available at `https://anonymous.4open.science/r/ICLR2027-Anonymous-Code-EE00`.

1 INTRODUCTION

Language-model agents are increasingly being developed to perform complex tasks in software environments (Luo et al., 2025). Terminal agents provide a general approach to this goal by using command-line interfaces to execute programs, coordinate tools, and complete multi-step workflows. Their potential applications extend from software engineering to scientific computing, engineering analysis, and other specialized domains (Yuan et al., 2026). Benchmarks such as SWE-bench, SWE-Gym, and Terminal-Bench evaluate their capabilities through executable outcomes (Jimenez et al., 2024; Pan et al., 2024; Merrill et al., 2026). However, existing training datasets concentrate on software engineering, with more limited coverage of other domains. Supporting these broader applications requires diverse training tasks with reliable checks of task completion

Constructing tasks in new domains is difficult because reference solutions and verifiers must account for domain-specific inputs, outputs, and correctness criteria. Verifiers must accept correct implementations with different algorithms or output representations while rejecting superficially valid results. Existing execution-based benchmarks rely on task-specific tests and validators (Hendrycks et al., 2021; Jain et al., 2025; Zhou et al., 2024). We therefore ask whether existing software workflows can provide executable reference solutions and generate the examples and expected outputs needed to construct and verify tasks across domains.

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

We introduce *software-in-the-loop reconstruction*, which creates training tasks by asking agents to reproduce the behavior of existing software. For each software workflow, we run several input configurations and record the outputs. We show the agent some input–output pairs and ask it to write a program without access to the original workflow's code. We then run the program on the remaining inputs and compare its outputs with those recorded from the original workflow. The workflow therefore serves as the reference solution and produces the expected outputs for verification. This provides supervision through software execution and reduces the need to write solutions and reference outputs separately for each task. Changing the input configurations creates multiple tasks from one workflow. Using workflows from different domains extends the same construction process to a broader range of tasks. This approach connects programming by example with behavioral system identification (Gulwani, 2011; Gulwani et al., 2017; Ljung et al., 1987). Verification still requires domain-specific comparison rules, whose reliability we assess through a separate audit after freezing the verifier and blinded human review.

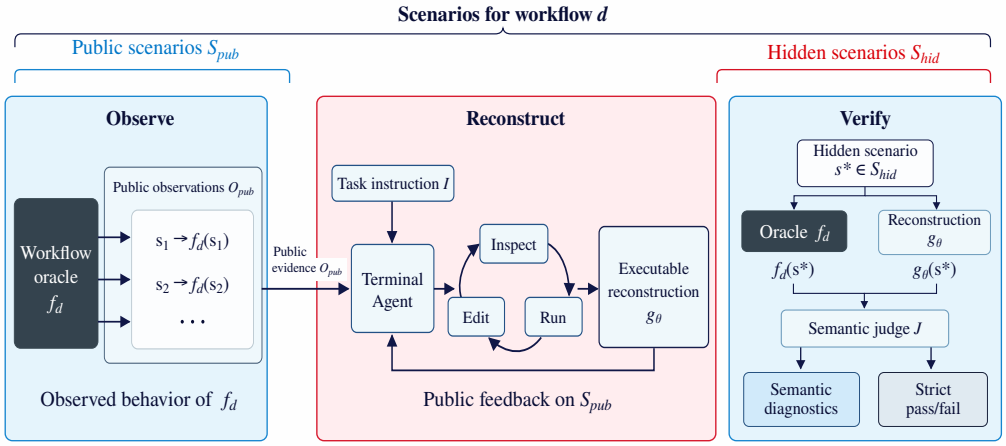


Figure 1: Software-in-the-loop reconstruction. Public observations of an executable workflow guide a terminal agent in constructing a reusable program.

Figure 1 illustrates how a software workflow supplies public evidence and hidden reference outputs for constructing reconstruction tasks, verifying candidate programs, and collecting training trajectories. We instantiate this procedure as SWR3000, comprising 3,000 tasks derived from 500 workflows across 46 software families. Qwen3.8-Max solves 22.8% of tasks in the first evaluation round and 27.9% at least once across three attempts, yielding 1,422 verified trajectories from 838 tasks. Many unsuccessful runs produce executable programs but fail the hidden behavioral checks, showing that artifact completion alone is insufficient for reconstruction success. To evaluate the training value of the verified trajectories, we construct an oversampled reconstruction-only corpus for supervised fine-tuning (SFT) of Qwen3.8-27B. We evaluate the resulting models on Terminal-Bench 2.0 (TB2), Terminal-Bench 4.0 (TB4), Long-Horizon Terminal-Bench (LHTB), and SWR100, using three seeds and matched-token corpus comparisons to assess the utility of reconstruction supervision under fixed experimental settings.

Our main contributions are as follows:

1. We formulate workflow-conditioned reconstruction for task construction across domains. Each executable workflow supplies a reference solution, public examples, and hidden reference outputs for multiple task variants, reducing the need to author solutions and verification targets separately.
2. We construct SWR3000 and a hierarchical semantic verifier. We assess task sensitivity, predefined replay controls, and verifier reliability, including an independent post-freeze audit.
3. We characterize reconstruction difficulty and evaluate verified trajectories as reconstruction-only SFT data, reporting three-seed results and comparisons with four matched-token corpus controls on downstream terminal-agent evaluations.

2 Related Work

Executable agent tasks and environment scaling. APPS and LiveCodeBench evaluate programs through execution (Hendrycks et al., 2021; Jain et al., 2025). InterCode, AgentBench, GAIA, WebArena, and OSWorld evaluate multi-step interaction in digital environments (Yang et al., 2023; Liu et al., 2024; Mialon et al., 2024; Zhou et al., 2024; Xie et al., 2024). SWE-bench, SWE-agent, SWE-Gym, OpenHands, and Terminal-Bench focus on software engineering and command-line agents (Jimenez et al., 2024; Yang et al., 2024; Pan et al., 2024; Wang et al., 2025; Merrill et al., 2026). Recent work scales terminal-agent environments, tasks, and trajectories (Raoof et al., 2026; Ivison et al., 2026b; Peng et al., 2026b; Pi et al., 2026b; Shen et al., 2026). We address task construction across domains by reusing workflows as reference solutions and generating expected outputs through execution. This reduces the need to author solutions and verification targets separately for each task.

Executable feedback and behavioral verification. The test-oracle problem concerns how to specify expected behavior (Barr et al., 2014). Differential testing compares implementations (McKeeman, 1998), while metamorphic testing checks relations under input transformations (Chen et al., 2020). ReAct, Reflexion, Self-Refine, Voyager, and CodeRL use interaction or execution feedback for iterative improvement (Yao et al., 2022; Shinn et al., 2023; Madaan et al., 2023; Wang et al., 2023; Le et al., 2022). Our hierarchical verifier compares candidate outputs with workflow outputs using domain-specific rules that accommodate valid output differences. Public checks guide repair, while final acceptance requires hidden behavioral agreement, structural compliance, and predefined anti-shortcut checks.

Program induction and system identification. Programming by example infers programs from input–output observations (Gulwani, 2011; Gulwani et al., 2017; Li et al., 2026; Huang et al., 2026). Neural and neuro-symbolic methods learn search procedures and reusable program abstractions (Devlin et al., 2017; Balog et al., 2016; Ellis et al., 2021; Reed & De Freitas, 2015). System identification learns predictive models from observations and interventions (Ljung et al., 1987; Brunton et al., 2016), while world models learn dynamics for prediction and planning (Ha & Schmidhuber, 2018; Hafner et al., 2019; Schrittwieser et al., 2020). We use this inference setting to construct terminal-agent training tasks. Agents reconstruct editable programs from public workflow examples and are evaluated on hidden inputs. Agreement on finite hidden scenarios does not establish recovery of the original implementation or mechanism.

3 Software-in-the-Loop Reconstruction

3.1 Task Definition

Constructing training tasks across domains requires reference solutions and reliable checks of candidate outputs. We reuse an existing software workflow as the reference solution and execute it on different inputs to obtain the expected outputs. The agent receives task instructions, an input schema, and public input–output examples. It must implement `reconstruct.py`, an editable program that reads a JSON-encoded input configuration and returns a JSON object. The agent can test its program on the public examples and use verifier feedback to revise it. Final evaluation runs the program on hidden inputs and compares its outputs with those produced by the workflow. The source workflow, hidden inputs, hidden reference outputs, and private verifier state remain unavailable to the agent. The task requires reproducing the specified behavior, without requiring recovery of the original source code.

Formally, we represent a software workflow d as an executable function f_d. A scenario s specifies one input configuration for the workflow, and $f_d(s)$ denotes the output produced by executing the workflow with that configuration. For each workflow, we construct a reconstruction task

$$\begin{gathered} T = (d, f_d, O_{\text{pub}}, S_{\text{pub}}, S_{\text{hid}}, J, I), \\ S_{\text{pub}} \cap S_{\text{hid}} = \emptyset, \qquad O_{\text{pub}} = \{(s, f_d(s)) : s \in S_{\text{pub}}\}. \end{gathered} \tag{1}$$

Here, S_{pub} and S_{hid} are disjoint public and hidden scenario sets, and O_{pub} contains the public input–output examples. The instruction is I, and J is the private semantic judge. The source workflow f_d

produces the reference outputs; the agent's editable program g_θ is evaluated against those outputs on hidden scenarios.

A failure on a hidden input. Hidden inputs can expose errors that the public examples do not reveal. For example, consider a task that repairs mass and inertia values in a URDF robot description. In one recorded run, Qwen3.8-Max produces a `reconstruct.py` program that passes all six public scenarios. On a hidden scenario, the program attempts to convert the categorical value `trajectory_target_7_hidden` into an integer, causing a `ValueError`. A GPT-5.6 Sol reconstruction handles this unseen value and passes the same task. The failure therefore concerns how the program handles a new input value, despite passing every public check. Appendix J provides the full comparison.

3.2 Workflow-Conditioned Task Generation

We reuse each workflow to construct six task variants by varying the public evidence and hidden evaluation scenarios. A typed scenario generator produces disjoint public and hidden scenario sets for each variant. Before release, each workflow must execute successfully, produce deterministic outputs with a stable schema, and show measurable behavioral variation across scenarios. The wrapped workflow, which adapts the source workflow to the reconstruction interface, must pass the verifier. All three predefined public-output replay controls must fail. These release gates yield SWR3000, comprising 3,000 tasks from 500 workflows. Variants of the same workflow should not be treated as independent software mechanisms. Appendix A provides the synthesis algorithm and interface details.

3.3 Hierarchical Semantic Verification

This subsection explains how we check whether a reconstructed program meets the task requirements. We describe the public feedback used for program revision and the checks required for final acceptance. Task T uses workflow d as its reference, with executable function f_d. The agent constructs program g_θ, where θ identifies its implementation. For an input scenario s, the judge J compares the candidate output $g_\theta(s)$ with the reference output $f_d(s)$. It checks mandatory output fields and task-specific invariants, meaning properties required of valid outputs. Different code and output serialization are allowed, but agreement on tested inputs does not establish general program equivalence.

The score $J(g_\theta(s), f_d(s)) \in [0, 1]$ is the fraction of mandatory checks satisfied. Execution or parsing failures receive zero. Let S_{pub} and S_{hid} denote the public and hidden scenario sets. Their average agreement scores are

$$r_x(T, g_\theta) = \frac{1}{|S_x|} \sum_{s \in S_x} J(g_\theta(s), f_d(s)), \qquad x \in \{\text{pub}, \text{hid}\}, \tag{2}$$

where $|S_x|$ is the number of scenarios in the set. We write these scores as $r_{\text{pub}} = r_{\text{pub}}(T, g_\theta)$ and $r_{\text{hid}} = r_{\text{hid}}(T, g_\theta)$. The public score guides program revision. The hidden score remains private, and $r_{\text{hid}} = 1$ requires every mandatory check to be satisfied on every hidden scenario.

Two binary indicators record additional requirements. $G_{\text{struct}} \in \{0, 1\}$ equals 1 when the program has editable source, a valid entry point, successful execution, and parseable output. $G_{\text{anti}} \in \{0, 1\}$ equals 1 when all predefined anti-shortcut checks are satisfied. These checks reject copied artifacts, public-example lookup replay, and constant outputs on tasks whose behavior varies. Each indicator is 0 otherwise.

The report $V(T, g_\theta)$ records all four results. Final acceptance requires complete hidden agreement and both indicators to equal 1:

$$\begin{aligned} V(T, g_\theta) &= (G_{\text{struct}}, r_{\text{pub}}, r_{\text{hid}}, G_{\text{anti}}), \\ \text{Pass}(T, g_\theta) &= \mathbf{1}\{G_{\text{struct}} = 1 \ \wedge \ r_{\text{hid}} = 1 \ \wedge \ G_{\text{anti}} = 1\}. \end{aligned} \tag{3}$$

Here, $\mathbf{1}\{\cdot\}$ equals 1 when all three conditions hold and 0 otherwise. The public score is not a condition for final acceptance.

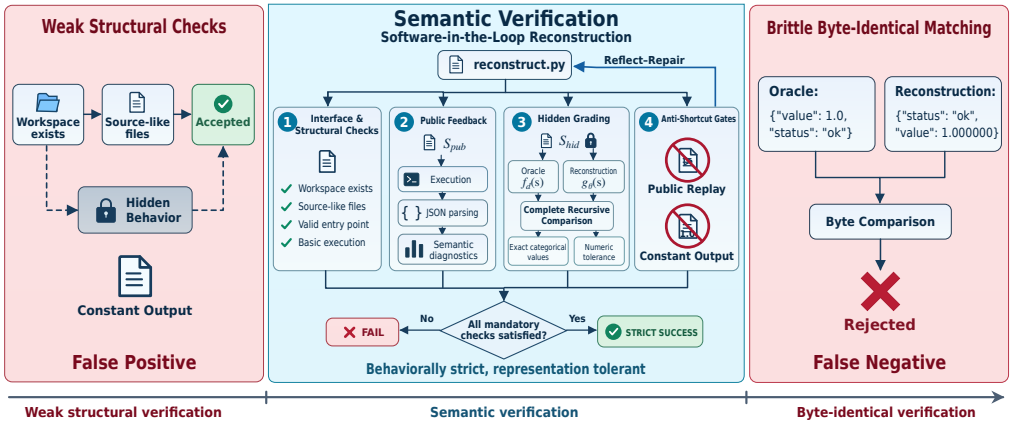


Figure 2: Hierarchical semantic verification. The public-feedback and hidden-grading stages compute r_{pub} and r_{hid}, respectively. Structural, hidden, and anti-shortcut gates determine strict success.

3.4 Reconstruction SFT

We retain an interaction trajectory for supervised fine-tuning (SFT) only when its final program satisfies the strict acceptance criterion. Let τ_i denote a recorded trajectory for task T_i, and let g_i denote its final reconstructed program. The eligible trajectory pool is

$$\mathcal{D}_{\text{SFT}} = \{\tau_i \mid \text{Pass}(T_i, g_i) = 1\}. \tag{4}$$

We exclude trajectories associated with evaluation task IDs, then sample training entries from the remaining pool. The resulting collection, $\mathcal{B}_{\text{SFT}}$, is a multiset because a trajectory may appear more than once. Each occurrence contributes to training, so repetition increases a trajectory's weight without adding an independent solution. We train the model with parameters ϕ to predict the recorded agent actions from their preceding interaction histories:

$$\mathcal{L}_{\text{SFT}}(\phi) = -\sum_{\tau \in \mathcal{B}_{\text{SFT}}} \sum_{t} \log p_\phi(a_t \mid h_t), \tag{5}$$

where t indexes agent steps, h_t is the interaction history before step t, and a_t is the recorded agent action. The probability $p_\phi(a_t \mid h_t)$ is assigned by the model to that action given the history. Each repeated trajectory is counted separately in the loss.

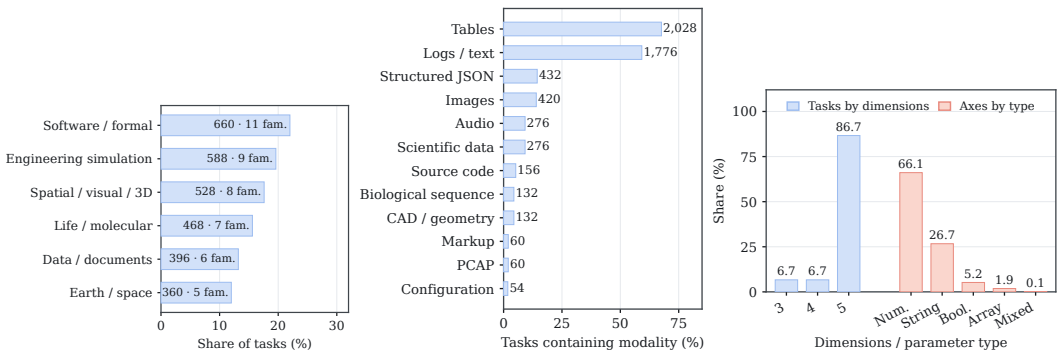
4 Experiments

The experiments evaluate software-in-the-loop reconstruction as a source of training data for terminal agents. We first examine whether the generated tasks require the intended behavior and whether the semantic verifier accepts valid outputs and rejects invalid ones. We then compare agents' ability to solve the reconstruction tasks. Finally, we evaluate whether fine-tuning on verified reconstruction trajectories improves performance on downstream terminal-agent benchmarks.

4.1 Experimental Setup

Task sets. We assess task validity and verifier behavior during development across all 3,000 SWR3000 tasks. The independent post-freeze audit covers 100 tasks. SWR100 contains 100 in-domain evaluation tasks whose task IDs are excluded from the final SFT corpus. It measures performance on held-out task instances and does not establish generalization to unseen workflows. The final reconstruction-SFT corpus contains no auxiliary terminal-task data and has no training-source overlap with TB2, TB4, or LHTB.

Agent evaluation. In the first full-suite round, we evaluate Qwen3.8-Max-0902 on all 3,000 tasks using Terminus-2. Each task receives one attempt, limited to 100 turns and one hour. We separately



(a) Balanced task and family coverage across six software domains.

(b) Artifact types exposed by workflows.

(c) Scenario parameter types and dimensions.

Figure 3: SWR3000 composition. (a) task distribution across software domains, (b) artifact types exposed by the workflows, and (c) scenario parameter types and dimensions.

evaluate eight API-served models on SWR100 with limits of 500 turns and three hours per task. The primary metric is strict hidden success. Partial semantic agreement supports diagnostic analysis only. Missing or unscored runs are counted as failures.

Repeated sampling and SFT. To measure success across repeated attempts and collect verified trajectories, we evaluate Qwen3.8-Max three times on each SWR3000 task. These attempts comprise the first full-suite round and two additional rounds using the same model and finalized verifier. The reported 22.8% Pass@1 is the first-round success rate, not an average across rounds. Pass@3 measures the fraction of tasks solved at least once across the three attempts. Let c_i denote the number of successful attempts on task i. Then

$$\text{Pass@3} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[c_i > 0],$$

where $N = 3000$ is the number of tasks and $\mathbf{1}[c_i > 0]$ equals 1 if task i is solved at least once and 0 otherwise. Missing attempts count as failures. For each reconstruction-SFT training budget, we train three independent checkpoints using seeds 42, 43, and 44. The corresponding Base runs use the same evaluation seeds. We report the mean and standard deviation across the three runs.

4.2 Behavioral Validity of SWR3000

We evaluate all 3,000 SWR3000 tasks to examine whether changing the inputs changes the workflow's outputs. We also test whether three methods can satisfy the hidden checks by returning outputs already provided in the public examples.

Coverage. SWR3000 contains 3,000 tasks derived from 500 workflows across 46 software families and six domains. No single software family accounts for more than 2.4% of the tasks. The tasks include 15,600 public scenarios and 8,400 hidden scenarios, each specifying an input configuration. Figure 3 summarizes the domain distribution, artifact types, and input parameters. Appendix B provides the full classification.

Table 1: Overview of SWR3000.

Dataset	Tasks	Families	Workflows	Public scenarios	Hidden scenarios
SWR3000	3,000	46	500	15,600	8,400

Sensitivity to input changes. For each task, we run the workflow with different input configurations and compare the resulting outputs. We record a change when at least one output field used to

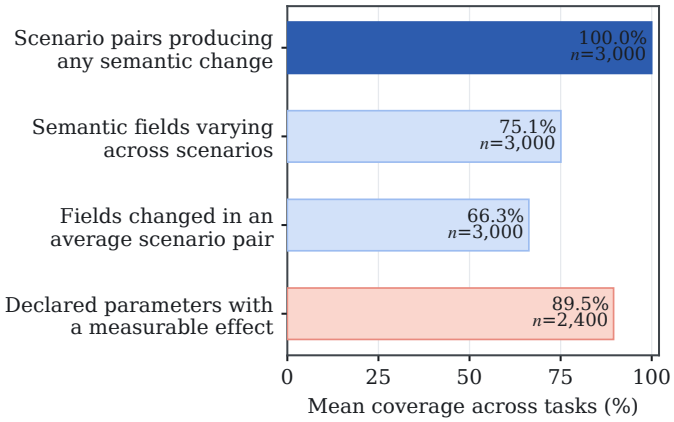


Figure 4: Behavioral sensitivity of SWR3000 to scenario interventions. The four metrics measure whether scenario pairs change any output, how many semantic fields vary across scenarios, how many fields change within an average scenario pair, and how many declared scenario parameters measurably affect workflow outputs.

assess task behavior differs between two executions. For 2,984 tasks, every compared pair produces a change. For each of the remaining 16 tasks, 35 of 36 pairs produce a change. Across all tasks, 75.1% of these output fields vary across input configurations. On average, two configurations differ in 66.3% of the fields. For 2,400 tasks, we can also change one input parameter while keeping the others fixed. Varying 89.5% of the declared parameters produces a measurable output change.

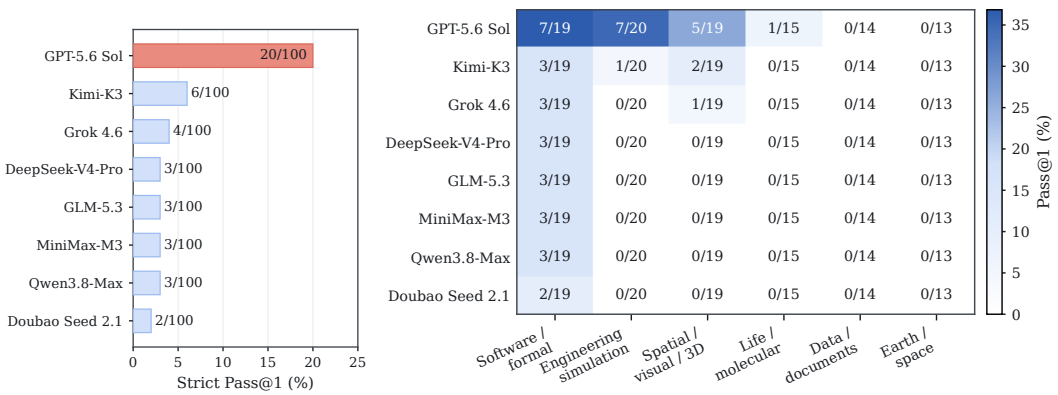
Public-output replay controls. We test three predefined methods that return a public output instead of reconstructing the workflow. The first returns the most frequent public output. The second returns the output from the public scenario whose inputs are closest to the hidden inputs. The third is an oracle control that uses the hidden reference output to select the best-matching public output. The controls achieve 34.3–44.3% agreement on hidden output fields, but none satisfies all strict acceptance requirements on any released task. Tasks are released only if all three controls fail these requirements. The zero-success result therefore confirms this selection rule and does not establish resistance to other shortcuts. Appendix B provides additional statistics on input changes and task construction checks.

4.3 Verifier Reliability

We assess whether the verifier accepts valid outputs and rejects invalid ones. During development, we construct 35,873 controlled test cases across all 3,000 tasks. These include outputs near numerical tolerance boundaries, structurally valid but incorrect outputs, outputs from the wrong scenario, and modified geometry outputs. We use these cases to adjust comparison tolerances and check verifier behavior before freezing its configuration. We assess the frozen verifier through a blinded human review of 200 cases drawn from the post-freeze audit, comprising 100 valid and 100 invalid cases. Against adjudicated human labels, the verifier accepts 2 invalid cases and rejects 2 valid cases, giving false acceptance and false rejection rates of 2.0% each. The review shows that the verifier distinguishes valid from invalid outputs in most examined cases. Appendix I provides the complete audit protocols, per-class results, and analysis of geometry-related errors.

4.4 Agent Performance

Full-suite evaluation. Qwen3.8-Max solves 685 of the 3,000 SWR3000 tasks in a single attempt, yielding 22.8% Pass@1. Valid evaluation records are available for 2,978 tasks, giving a success rate of 23.0% on this subset. We report the full-suite rate using all 3,000 tasks as the denominator and count missing records as failures.



(a) Strict Pass@1 over all 100 scheduled tasks.

(b) Strict successes by domain; each cell reports passed/scheduled tasks.

Figure 5: Eight-model SWR100 comparison. (a) overall success and (b) domain-level performance.

Failure structure. Among the 2,781 evaluated workspaces containing `reconstruct.py`, 1,591 return the same output for different hidden scenarios, and 75 directly replay public outputs. These results show that producing a reconstruction file does not ensure that the program reproduces the required behavior on hidden inputs. Successful runs finish in a median of 58 turns, compared with 85 turns for failures attributed to the model. This comparison is observational and does not establish that longer interaction causes failure. Appendix D provides the diagnostic accounting.

Table 2: Single-attempt Qwen3.8-Max evaluation on SWR3000 with the finalized verifier. Pass@1 uses all 3,000 scheduled tasks.

Scheduled	Records	Strict pass	Pass@1	Record rate	Pass/record
3,000	2,978	685	22.8%	99.3%	23.0%

Eight-model comparison. We evaluate eight API-served models on the same 100 SWR100 tasks under a shared interaction budget. GPT-5.6 Sol achieves 20.0% Pass@1, while the other models achieve between 2.0% and 6.0% (Figure 5). Across the eight models, 43 of the 44 successful runs occur in Software & Formal Systems, Engineering Simulation, or Spatial, Visual & 3D. No successful run is observed in Data & Documents or Earth & Space. SWR100 therefore remains challenging for the evaluated models, with successful reconstructions concentrated in three domains.

4.5 Reconstruction SFT

We next evaluate whether fine-tuning on verified reconstruction trajectories improves performance on held-out reconstruction tasks and external terminal-agent benchmarks.

Verified trajectory collection. We run Qwen3.8-Max three times on each SWR3000 task. At least one attempt succeeds on 838 of the 3,000 tasks, yielding 27.9% Pass@3. These attempts produce 1,422 verified trajectories as the source pool for reconstruction SFT. We exclude evaluation task IDs before sampling training entries. Appendices C and E provide the repeated-attempt and corpus-construction details.

Table 3: Final three-attempt Qwen3.8-Max result on SWR3000.

Result	Verified tasks/trajectories	Rate
Pass@3 / task union	838	27.9%
All verified trajectories	1,422	–

Training-entry budget. We fine-tune Qwen3.8-27B with 750, 1,500, and 3,000 reconstruction training entries, using three training seeds per budget. These budgets count sampled entries, including repeated trajectories. With 3,000 entries, mean performance increases from 47.94% to 53.56% on TB2, from 0.51% to 3.54% on TB4, from 20.67% to 27.67% on LHTB, and from 1.33% to 3.33% on SWR100. For each evaluation, the mean is non-decreasing across the tested budgets. These results support increasing the number of training entries under the specified sampling procedure.

Table 4: Qwen3.8-27B reconstruction-SFT training-entry budgets, including repeated samples (mean $\pm$ standard deviation over seeds 42, 43, and 44; each SFT seed is separately trained and tested; all entries are percentages).

Training entries	TB2	TB4	LHTB reward	SWR100 success
Base	47.94 ± 1.30	0.51 ± 0.87	20.67 ± 1.15	1.33 ± 0.58
SFT-750	50.19 ± 0.65	1.01 ± 0.87	22.33 ± 0.58	1.33 ± 0.58
SFT-1500	50.56 ± 1.12	2.02 ± 0.87	24.33 ± 1.53	2.67 ± 0.58
SFT-3000	53.56 ± 0.65	3.54 ± 0.87	27.67 ± 0.58	3.33 ± 0.58

Matched-token controls. We compare reconstruction supervision with four other agent-training corpora under a matched training-token budget. OpenThoughts-Agent-v1 provides curated trajectories for general-purpose agents (Raoof et al., 2026). TMax constructs terminal-agent training data using a structured task taxonomy and environments with different verifiers (Ivison et al., 2026a). LiteCoder-Terminal generates synthetic, executable long-horizon terminal environments and expert trajectories (Peng et al., 2026a). Nemotron-Terminal combines adapted existing tasks with skill-based synthetic task generation (Pi et al., 2026a). Reconstruction SFT and all four controls use the same budget of 88.48M assistant-loss tokens. Reconstruction SFT achieves the highest mean performance on all four evaluations. Its advantage therefore persists when the assistant-loss-token budget is matched.

Table 5: Matched-token Qwen3.8-27B corpus comparison (mean $\pm$ standard deviation over three seeds; evaluation entries are percentages). Every fine-tuned model receives 88.48M assistant-loss tokens. The reconstruction corpus contains 3,000 sampled entries, including repeats.

Training corpus	Loss tokens	TB2	TB4	LHTB	SWR100
Base (no SFT)	–	47.94 ± 1.30	0.51 ± 0.87	20.67 ± 1.15	1.33 ± 0.58
OpenThoughts-Agent-v1	88.48M	49.06 ± 1.30	0.00 ± 0.00	22.67 ± 1.53	1.67 ± 0.58
TMax	88.48M	49.44 ± 2.25	1.01 ± 0.87	20.67 ± 1.53	1.33 ± 0.58
LiteCoder-Terminal	88.48M	48.69 ± 1.72	0.51 ± 0.87	23.00 ± 1.00	1.00 ± 1.00
Nemotron-Terminal	88.48M	47.19 ± 1.12	0.51 ± 0.87	23.67 ± 1.15	1.33 ± 1.53
Reconstruction (ours)	88.48M	$\mathbf{53.56 \pm 0.65}$	$\mathbf{3.54 \pm 0.87}$	$\mathbf{27.67 \pm 0.58}$	$\mathbf{3.33 \pm 0.58}$

5 Conclusion

We introduced software-in-the-loop reconstruction to construct terminal-agent training tasks across domains by reusing existing software workflows. Each workflow serves as an executable reference solution and generates public examples and hidden reference outputs under different input configurations. This reduces the need to author solutions and verification targets separately for each task. We instantiate the procedure as SWR3000, comprising 3,000 tasks derived from 500 workflows across 46 software families. Supervised fine-tuning on verified reconstruction trajectories improves mean downstream terminal-agent performance across three seeds. Under a matched assistant-loss-token budget, reconstruction SFT achieves the highest mean performance on all four evaluations among the compared training corpora. These results support workflow reuse for task construction across domains and the collection of verified supervision for terminal-agent training. Future work can assess generalization to unseen workflows and separate the contribution of additional unique trajectories from that of repeated sampling.

AI USE STATEMENT

Generative AI tools assisted with writing, editing, and code transformation. The authors remain responsible for the scientific claims, experimental results, and released artifacts.

ETHICS STATEMENT

SWR3000 uses bounded terminal environments. Release audits exclude secrets, credentials, proprietary artifacts, and operational instructions that could enable misuse, with additional review for security- and infrastructure-adjacent workflows.

REPRODUCIBILITY STATEMENT

The anonymous code repository contains task-generation code, metadata, verifier specifications, evaluation scripts, and the 100-task SWR100 evaluation set. The appendix documents the reconstruction interface, dataset audit, evaluation protocol, aggregate run accounting, SFT corpus construction, and per-seed scaling results.

REFERENCES

Matej Balog, Alexander L Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. Deepcoder: Learning to write programs. *arXiv preprint arXiv:1611.01989*, 2016.

Earl T Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE transactions on software engineering*, 41(5):507–525, 2014.

Steven L Brunton, Joshua L Proctor, and J Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the national academy of sciences*, 113(15):3932–3937, 2016.

Tsong Y Chen, Shing C Cheung, and Shiu Ming Yiu. Metamorphic testing: a new approach for generating next test cases. *arXiv preprint arXiv:2002.12543*, 2020.

Jacob Devlin, Jonathan Uesato, Surya Bhupatiraju, Rishabh Singh, Abdel-rahman Mohamed, and Pushmeet Kohli. Robustfill: Neural program learning under noisy i/o. In *International conference on machine learning*, pp. 990–998. PMLR, 2017.

Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B Tenenbaum. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*, pp. 835–850, 2021.

Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. *ACM Sigplan Notices*, 46(1):317–330, 2011.

Sumit Gulwani, Oleksandr Polozov, and Rishabh Singh. Program synthesis. *Foundations and trends in programming languages*, 4(1-2):1–119, 2017.

David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3):440, 2018.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International conference on machine learning*, pp. 2555–2565. PMLR, 2019.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.

Jingyuan Huang, Zuming Huang, Yucheng Shi, Tianze Yang, Xiaoming Zhai, Wei Chu, and Ninghao Liu. Trust the right teacher: Quality-aware self-distillation for gui grounding. *arXiv preprint arXiv:2606.18101*, 2026.

Hamish Ivison, Junjie Oscar Yin, Rulin Shao, Teng Xiao, Nathan Lambert, and Hannaneh Hajishirzi. Tmax: A simple recipe for terminal agents. *arXiv preprint arXiv:2606.23321*, 2026a.

Hamish Ivison, Junjie Oscar Yin, Rulin Shao, Teng Xiao, Nathan Lambert, and Hannaneh Hajishirzi. Tmax: A simple recipe for terminal agents. *arXiv preprint arXiv:2606.23321*, 2026b.

Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, pp. 58791–58831, 2025.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pp. 54107–54157, 2024.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems*, 35:21314–21328, 2022.

Zhongzhi Li, Xuansheng Wu, Yijiang Li, Lijie Hu, and Ninghao Liu. Less is enough: synthesizing diverse data in feature space of llms. *arXiv e-prints*, pp. arXiv–2602, 2026.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pp. 52989–53046, 2024.

Lennart Ljung et al. Theory for the user. *System identification*, 45, 1987.

Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, et al. Large language model agent: A survey on methodology, applications and challenges. *arXiv preprint arXiv:2503.21460*, 2025.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.

William M McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.

Mike Merrill, Alexander Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Shin, Thomas Walshe, E Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *International Conference on Learning Representations*, volume 2026, pp. 40903–40986, 2026.

Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. In *International Conference on Learning Representations*, volume 2024, pp. 9025–9049, 2024.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. *arXiv preprint arXiv:2412.21139*, 2024.

Xiaoxuan Peng, Kaiqi Zhang, Xinyu Lu, Boxi Cao, Yaojie Lu, Hongyu Lin, Xianpei Han, and Le Sun. Litecoder-terminal: Scaling long-horizon terminal environments for learning language agents. *arXiv preprint arXiv:2605.29559*, 2026a.

Xiaoxuan Peng, Kaiqi Zhang, Xinyu Lu, Boxi Cao, Yaojie Lu, Hongyu Lin, Xianpei Han, and Le Sun. Litecoder-terminal: Scaling long-horizon terminal environments for learning language agents. *arXiv preprint arXiv:2605.29559*, 2026b.

Renjie Pi, Grace Lam, Mohammad Shoeybi, Pooya Jannaty, Bryan Catanzaro, and Wei Ping. On data engineering for scaling llm terminal capabilities. *arXiv preprint arXiv:2602.21193*, 2026a.

Renjie Pi, Grace Lam, Mohammad Shoeybi, Pooya Jannaty, Bryan Catanzaro, and Wei Ping. On data engineering for scaling llm terminal capabilities. *arXiv preprint arXiv:2602.21193*, 2026b.

Negin Raoof, Richard Zhuang, Marianna Nezhurina, Etash Guha, Atula Tejaswi, Ryan Marten, Charlie F Ruan, Tyler Griggs, Alexander Glenn Shaw, Hritik Bansal, et al. Openthoughts-agent: Data recipes for agentic models. *arXiv preprint arXiv:2606.24855*, 2026.

Scott Reed and Nando De Freitas. Neural programmer-interpreters. *arXiv preprint arXiv:1511.06279*, 2015.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.

Qijia Shen, Zhiqi Huang, Vamsidhar Kamanuru, Aznaur Aliev, Jay Rainton, Ahmed Awelkair, Zhichen Zeng, Jiajun Li, Shi Dong, Yueming Yuan, et al. Seta: Scaling environments for terminal agents. *arXiv preprint arXiv:2607.10891*, 2026.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pp. 65882–65919, 2025.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.

John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. Intercode: Standardizing and benchmarking interactive coding with execution feedback. *Advances in Neural Information Processing Systems*, 36:23826–23854, 2023.

John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

Xiaoyang Yuan, Haoxi Zeng, Wencheng Ye, Yi Bin, Wenqi Shao, Chen Qian, Wei Ye, Yujuan Ding, Zheng Wang, Pengpeng Zeng, et al. Terminal agents: A survey of ai agents in command-line environments. 2026.

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pp. 15585–15606, 2024.

APPENDIX

This appendix provides additional methodological details, implementation information, and experimental results. Sections A–J are organized as follows.

- **A. Benchmark Specification.** Appendix A describes the task-generation procedure, reconstruction interface, release checks, semantic scoring, public feedback, and final evaluation protocol.
- **B. Dataset Composition and Audit.** Appendix B summarizes the domain and software-family coverage of SWR3000, the artifact types and scenario parameters used in the tasks, behavioral sensitivity to scenario changes, public-output replay baselines, and remaining dataset limitations.
- **C. Repeated-Attempt Pass@3 Analysis.** Appendix C reports Pass@3 over three attempts, the number of tasks solved across repeated runs, and the software families that remain unsolved.
- **D. Full-Suite Diagnostics.** Appendix D provides detailed results for the full SWR3000 evaluation, including missing and infrastructure-failed runs, strict successes, common failure modes, interaction lengths, and family-level difficulty.
- **E. Reconstruction SFT Corpus Construction.** Appendix E describes how verified reconstruction trajectories are converted into the final SFT corpus through oversampling, within-trajectory cleanup, conversation repair, and length filtering.
- **F. Reconstruction Fine-Tuning Scaling and Controls.** Appendix F reports the complete per-seed results for Base, SFT-750, SFT-1500, and SFT-3000, together with matched-token comparisons against OpenThoughts-Agent, TMax, LiteCoder-Terminal, and Nemotron-Terminal.
- **G. SWR100 Model and Domain Results.** Appendix G provides the complete eight-model results on SWR100 and shows how successful tasks are distributed across software domains.
- **H. SWR100 Interaction and Statistical Diagnostics.** Appendix H analyzes interaction length, token usage, scoring completeness, and the uncertainty introduced by evaluating each model–task pair only once.
- **I. Verifier Mutation Stress Tests.** Appendix I evaluates the verifier on controlled output modifications, including tolerance-boundary cases, missing input-dependent behavior, wrong-scenario outputs, semantic perturbations, and geometry changes.
- **J. Failure Signatures and Matched Trajectory Cases.** Appendix J analyzes failure modes across all 800 SWR100 model–task pairs and compares trajectories from different models on the same tasks using private-verifier feedback.

A BENCHMARK SPECIFICATION

Task materialization. Each of the 500 source workflows is parameterized by a typed scenario schema and produces six reconstruction variants, yielding 3,000 tasks. Before agent interaction begins, scenario generators sample disjoint public and hidden parameter settings, execute the source workflow, and materialize the resulting artifacts. In total, the release contains 15,600 public scenarios (5.2 per task on average) and 8,400 hidden scenarios (2.8 per task). The agent receives the instruction, scenario schema, public scenarios, and public execution artifacts, but never receives the source workflow, hidden inputs, hidden references, or private verifier state.

Task-synthesis algorithm. Algorithm 1 gives the complete task-generation procedure used to construct SWR3000 from the 500 source workflows. For a fixed workflow d, superscript (k) denotes its k-th task variant. Thus, $T_{d,k}$ instantiates the task T defined in Eq. 1. The wrapped reference program $g_d^\star$ implements the reconstruction interface and is distinct from the agent-generated program g_θ. Each $c \in \mathcal{C}_{\mathrm{pub}}$ is a replay control evaluated through the same input–output interface. The best-public control uses the hidden reference to select among public outputs. Every call to Pass applies the strict acceptance criterion in Eq. 3. Before a task is added to the dataset, the wrapped source workflow must solve it, and the three predefined public-output replay controls must fail. This programmatic process does not use an LLM to generate task instances.

Executable interface. Each task requires the agent to implement `reconstruct.py`, which reads one scenario encoded as JSON and returns one JSON object. All software families use the same interface, so a common execution harness can evaluate heterogeneous workflows. During interaction, the program is executed only on public scenarios and receives semantic feedback on those outputs. Final evaluation runs the same program on hidden scenarios that are never exposed to the agent. The implementation is unrestricted as long as it remains editable and follows the required interface.

Release gates. A generated task is retained only after passing a fixed set of validity checks. The source workflow must execute successfully, produce deterministic outputs across repeated runs, preserve a stable output schema, and use disjoint public and hidden scenarios. The selected scenarios must also produce measurable changes in workflow behavior. Finally, the wrapped source workflow must pass the verifier, while predefined public-output replay strategies must fail. These checks ensure that each released task has a valid behavioral target and rejects the predefined public-output replay strategies.

Evaluation separation. SWR100 is a held-out in-domain evaluation set: all 100 task identities are excluded from the final reconstruction-SFT corpus. This separation supports evaluation on held-out task instances, not a claim of unseen-workflow generalization. TB2, TB4, and LHTB are external evaluations with no training-source overlap. The final reconstruction-SFT runs do not mix in auxiliary terminal-task conversations.

Strict and diagnostic scores. Final success is determined only by hidden evaluation. A submission must provide a valid executable reconstruction, produce schema-compatible outputs, match all required hidden behaviors, and pass the anti-shortcut checks. For 2,940 tasks, outputs are compared recursively over all mandatory fields. The remaining 60 geometry tasks use a domain-specific comparator. Categorical values, Booleans, and integers require exact agreement, while floating-point values use absolute and relative tolerances of 10^{-6} and 10^{-4}. Partial semantic agreement is reported for diagnosis but does not affect the binary success decision.

Public feedback and final accounting. During interaction, the agent receives feedback only from public scenarios. This feedback reports whether `reconstruct.py` executes correctly, produces valid outputs, and matches the observed public behavior. Hidden scenarios and their reference outputs remain private until final grading. A reconstruction can therefore perform well on public examples while still failing on unseen hidden cases. Infrastructure failures are recorded separately, while the primary Pass@1 metric is computed over all scheduled tasks.

Algorithm 1 Workflow-Conditioned Reconstruction Task Synthesis

Require: Executable workflows $\{(d, f_d)\}_{d=1}^{500}$, typed scenario generators, and $K = 6$ variants per workflow
Ensure: Reconstruction dataset $\mathcal{D}_{\text{SWR3000}}$

$\mathcal{D}_{\text{SWR3000}} \leftarrow \emptyset$
for each workflow (d, f_d) **do**
  for $k = 1$ to K **do**
    repeat
  // Stage 1: Scenario construction and workflow execution
      $(S_{\text{pub}}^{(k)}, S_{\text{hid}}^{(k)}) \leftarrow \textsc{SampleScenarios}(d, k)$
      $valid \leftarrow [S_{\text{pub}}^{(k)} \cap S_{\text{hid}}^{(k)} = \emptyset]$
      if $valid$ **then**
        Execute f_d twice on $S_{\text{pub}}^{(k)} \cup S_{\text{hid}}^{(k)}$
        $valid \leftarrow \textsc{Successful}(f_d) \wedge \textsc{Deterministic}(f_d) \wedge \textsc{SchemaStable}(f_d) \wedge \textsc{InterventionSensitive}(f_d)$
      end if
      if $valid$ **then**
        $O_{\text{pub}}^{(k)} \leftarrow \{(s, f_d(s)) : s \in S_{\text{pub}}^{(k)}\}$
        Retain $\{(s, f_d(s)) : s \in S_{\text{hid}}^{(k)}\}$ as private references
      end if
  // Stage 2: Task materialization
      if $valid$ **then**
        $I^{(k)} \leftarrow \textsc{RenderInstruction}(d, O_{\text{pub}}^{(k)})$
        $J^{(k)} \leftarrow \textsc{BuildSemanticJudge}(f_d, S_{\text{hid}}^{(k)})$
        $T_{d,k} \leftarrow (d, f_d, O_{\text{pub}}^{(k)}, S_{\text{pub}}^{(k)}, S_{\text{hid}}^{(k)}, J^{(k)}, I^{(k)})$
        $\textsc{Materialize}(T_{d,k}$, environment, public feedback, private verifier)
      end if
  // Stage 3: Oracle and anti-shortcut gates
      if $valid$ **then**
        $g_d^\star \leftarrow \textsc{WrapWorkflow}(f_d)$
        $\mathcal{C}_{\text{pub}} \leftarrow \{$modal replay, nearest replay, best-public replay$\}$
        $valid \leftarrow [\text{Pass}(T_{d,k}, g_d^\star) = 1]$
        for each shortcut control $c \in \mathcal{C}_{\text{pub}}$ **do**
          if $\text{Pass}(T_{d,k}, c) = 1$ **then**
            $valid \leftarrow$ false
          end if
        end for
      end if
    until $valid$
    $\mathcal{D}_{\text{SWR3000}} \leftarrow \mathcal{D}_{\text{SWR3000}} \cup \{T_{d,k}\}$
  end for
end for
assert $|\mathcal{D}_{\text{SWR3000}}| = 500 \times 6 = 3000$
return $\mathcal{D}_{\text{SWR3000}}$

Table 6: Evaluation and repeated-sampling protocols.

Study	Tasks	Runs/model	Turn limit	Wall time	Retained outcomes
Full-suite evaluation	3,000	1	100	1 hour	Success and failure
Three-attempt Pass@3	3,000	3	–	–	Final task union
SWR100 comparison	100	1	500	3 hours	Success and failure
SFT scaling	4 suites	3	–	–	Train-and-test seeds

Training and decoding configuration. We initialize Qwen3.8-27B from the official Hugging Face snapshot `Qwen/Qwen3.8-27B`, which we convert once to a matching-layout Megatron `torch_dist` checkpoint rather than loading it through the Hugging Face bridge. Full-parameter SLIME SFT runs on eight nodes with eight NVIDIA GPUs per node (64 GPUs total), tensor parallelism 4, pipeline parallelism 2, context parallelism 2, sequence parallelism, and global batch size 128. The maximum context length is 262,145 tokens and dynamic packing is capped at 49,152 tokens per GPU. Thinking is disabled. The sample-level `sft_loss` uses `loss_mask_type=qwen3_5`, so only assistant tokens contribute to the objective. Adam uses $(\beta_1, \beta_2) = (0.9, 0.98)$, cosine decay from 1×10^{-5} to 1×10^{-6}, 5% warmup, and zero weight decay. Optimizer and RNG states are neither loaded nor saved. After chat-template rendering, rows longer than 128k tokens are dropped rather than split with a sliding window. The SWR100 API runs use temperature 0.2, top-p 0.95, at most 8,192 output tokens per call, at most 500 agent turns, and a three-hour task limit.

B Dataset Composition and Audit

Domain and family coverage. SWR3000 covers 46 software families grouped into six application domains. Software & Formal Systems contains 11 families and 660 tasks, followed by Physical & Engineering Simulation with 9 families and 588 tasks, Spatial, Visual & 3D with 8 families and 528 tasks, Life & Molecular Sciences with 7 families and 468 tasks, Data & Document Workflows with 6 families and 396 tasks, and Earth & Space Sciences with 5 families and 360 tasks. No single software family accounts for more than 2.4% of the benchmark, limiting the influence of any individual software family. Figure 6 shows the complete domain-to-family hierarchy.

Observable artifact diversity. Artifact labels are multi-valued because a single task may contain several output types. Tables appear in 2,028 tasks and logs or text in 1,776 tasks. The benchmark also includes structured JSON (432), images (420), audio (276), scientific data (276), source code (156), biological sequences (132), geometry (132), markup (60), packet captures (60), and configuration files (54).

Intervention complexity. Most tasks use five parameters (86.7%), while three- and four-parameter scenarios each account for 6.7% of the benchmark. Across all parameter axes, numeric values are most common (66.1%), followed by strings (26.7%), Booleans (5.2%), arrays (1.9%), and mixed types (0.1%).



Figure 6: Domain and software-family composition of SWR3000.

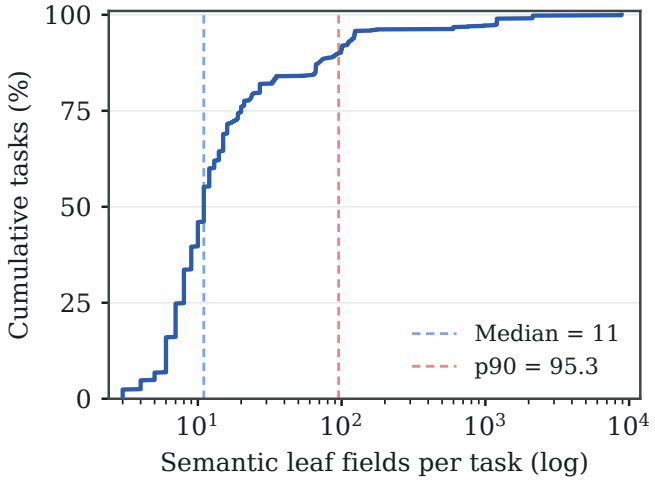


Figure 7: Distribution of semantic output size.

Semantic output complexity. For each task, we count the output fields that can be meaningfully checked by the verifier. After excluding digest-like fields used only for profiling, the median task contains 11 such fields, and the 90th percentile is 95.3 fields. The larger outputs mainly come from workflows that produce nested tables, geometry descriptors, or scientific arrays. Because strict grading checks every required field, the reconstruction must match many related output values simultaneously.

Behavioral grounding and shortcut resistance. We also check whether workflow outputs change with the input scenario and whether hidden cases can be solved by replaying public outputs. All 3,000 tasks change under at least one scenario comparison, and 2,984 change for every observed scenario pair. On average, 75.1% of output fields vary across scenarios, and 66.3% of fields change within a scenario pair. For the 2,400 tasks where input parameters can be changed one at a time, 89.5% of parameters produce a measurable output change. We further evaluate three public-output replay baselines in Table 7. The strongest baseline, oracle-selected best-public replay, reaches 44.3% hidden-field agreement, while nearest-scenario replay and constant-output replay reach 38.0% and 34.3%, respectively. None achieves a strict pass on any task. These results confirm that the three evaluated public-output replay strategies do not solve the released tasks, consistent with the release criterion.

Table 7: Public-output shortcut audit on SWR3000.

Control	Hidden-field agreement	Strict success
Oracle	100.0%	100.0%
Oracle-selected best-public replay	44.3%	0.0%
Nearest public scenario	38.0%	0.0%
Constant public output	34.3%	0.0%

Within-workflow variant audit. Each of the 500 workflows produces six task variants that share the same underlying executable workflow. Public examples may be reused across variants to provide sufficient observations. The six variants therefore do not represent independent software mechanisms or independent sets of demonstrations. Their hidden evaluations, however, remain separate. We audit all 15 pairs of variants within each workflow, giving 7,500 pairwise comparisons. Hidden scenario sets are disjoint across sibling variants, and no hidden scenario appears in the public or hidden set of another sibling. Each workflow also produces six distinct sets of hidden outputs. Thus, the six variants share the same underlying mechanism but differ in the hidden scenarios and behavioral targets used for evaluation.

Dataset limitations. SWR3000 inherits some regularities from its workflow-based construction. Related tasks may share similar instruction patterns or workflow structure, and task difficulty varies across software families.

C Repeated-Attempt Pass@3 Analysis

Evaluation setup. The full-suite evaluation runs Qwen3.8-Max once on each SWR3000 task. We evaluate two additional attempts per task using the same finalized verifier. A task is counted as solved under Pass@3 if at least one of the three attempts succeeds. Successful attempts are retained as verified trajectories for reconstruction SFT.

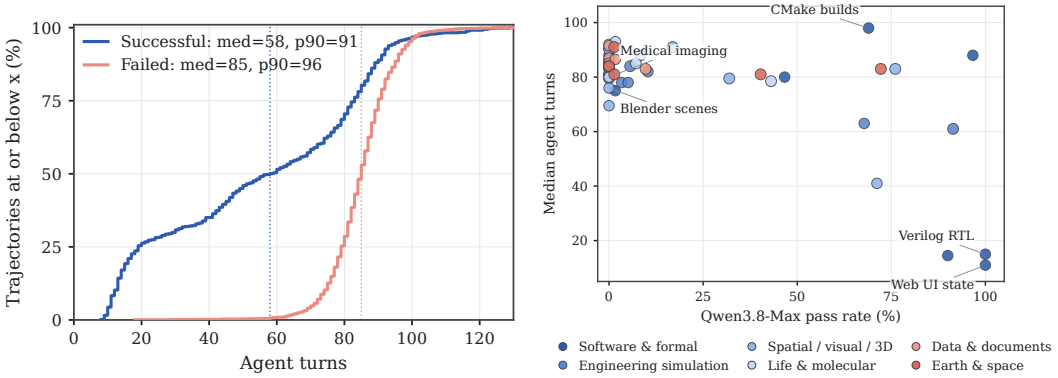
Pass@3 results. Across three attempts, 838 of the 3,000 tasks are solved at least once, yielding 27.9% Pass@3. Among these tasks, 383 succeed in exactly one attempt, 326 in two attempts, and 129 in all three attempts. Thus, 455 of the 838 solved tasks succeed more than once. In total, the successful attempts produce 1,422 verified trajectories for reconstruction SFT.

Family-level results. Repeated sampling increases the number of solved tasks, but substantial difficulty remains in several software families. After three attempts, 20 of the 46 families contain no solved task. This suggests that additional sampling alone does not resolve all of the challenges represented in SWR3000.

D Full-Suite Diagnostics

Failure analysis. The finalized evaluation records 685 strict passes among 3,000 tasks, corresponding to 22.8% Pass@1. Among the 2,781 evaluated workspaces that contain `reconstruct.py`, 1,591 return the same output across hidden scenarios and 75 replay released outputs. Thus, many agents produce an executable reconstruction but fail to capture how workflow outputs change with the input scenario. Failed runs still achieve 75.4% mean diagnostic agreement, indicating that many reconstructions match part of the hidden behavior without satisfying all requirements for strict success.

Interaction cost. Successful reconstructions terminate in a median of 58 turns, compared with 85 turns for model-attributable failures. Figure 8a shows that failed runs generally require longer interactions rather than differing only because of a small number of extreme cases. Across software families, pass rate is negatively correlated with median interaction length ($r = -0.647$; Figure 8b). Families with lower success rates therefore also tend to require longer agent interactions.



(a) Successful reconstructions terminate earlier than model-attributable failures in the finalized-verifier single-attempt evaluation.

(b) Family pass rate versus median turns in the finalized-verifier single-attempt evaluation.

Figure 8: Interaction diagnostics: (a) trajectory-length distributions and (b) family-level difficulty versus interaction cost.

E SFT CORPUS CONSTRUCTION

Corpus construction. Uniform re-verification yields 1,422 successful trajectories from 838 reconstruction tasks. The final reconstruction-SFT corpus contains 3,000 sampled training entries obtained by oversampling verified trajectories, primarily repeating trajectories from tasks with only one successful attempt. Repeated entries reuse the same successful interaction and do not increase the number of independent trajectories. Evaluation task identities are excluded from training. We remove duplicate events and post-solution actions within conversations and repair conversation boundaries. This within-trajectory cleanup is distinct from intentional repetition of complete training entries. The final corpus contains only reconstruction data, with no auxiliary terminal-task mixture.

Training data and optimization. Each trajectory is rendered independently for the final from-base run. Rows longer than 128k tokens are dropped, and 49,152 tokens is used only for per-GPU dynamic packing. The training configuration is described in Appendix A.

F RECONSTRUCTION FINE-TUNING SCALING AND CONTROLS

Per-seed results. Table 8 provides the per-seed results underlying Table 4. The 3,000-entry checkpoint outperforms the Base model for every seed on all four benchmarks. Results at the intermediate budgets show a progression in performance as the sampled training-entry budget increases. The size of the gains differs across benchmarks.

Table 8: Per-seed Qwen3.8-27B training-entry-budget results underlying Table 4. LHTB and SWR100 are reported as percentages.

Model	Seed	TB2 (%)	TB4 (%)	LHTB	SWR100
Base	42	47.19	0.00	22.00	1.00
Base	43	49.44	1.52	20.00	1.00
Base	44	47.19	0.00	20.00	2.00
SFT-750	42	50.56	1.52	23.00	1.00
SFT-750	43	49.44	1.52	22.00	2.00
SFT-750	44	50.56	0.00	22.00	1.00
SFT-1500	42	51.69	1.52	24.00	2.00
SFT-1500	43	50.56	1.52	23.00	3.00
SFT-1500	44	49.44	3.03	26.00	3.00
SFT-3000	42	53.93	3.03	28.00	3.00
SFT-3000	43	52.81	4.55	27.00	4.00
SFT-3000	44	53.93	3.03	28.00	3.00

Matched-token corpus controls. Table 5 compares reconstruction supervision with four alternative agent-training corpora. Each fine-tuned checkpoint receives 88.48M assistant-loss tokens, meaning tokens whose prediction contributes to the supervised loss. This is the common budget used in the comparison. For reconstruction SFT, 3,000 denotes sampled training entries including intentional repetition, not unique source trajectories. We do not compare cross-corpus unique-trajectory or exposure counts because the available counts use different native accounting conventions.

Table 9: Per-seed results for the four matched-token control corpora. All evaluation entries are percentages.

Training corpus	Seed	TB2	TB4	LHTB	SWR100
OpenThoughts-Agent-v1	42	48.31	0.00	21.00	1.00
OpenThoughts-Agent-v1	43	50.56	0.00	23.00	2.00
OpenThoughts-Agent-v1	44	48.31	0.00	24.00	2.00
TMax	42	49.44	0.00	21.00	2.00
TMax	43	47.19	1.52	19.00	1.00
TMax	44	51.69	1.52	22.00	1.00
LiteCoder-Terminal	42	50.56	0.00	22.00	2.00
LiteCoder-Terminal	43	48.31	0.00	24.00	0.00
LiteCoder-Terminal	44	47.19	1.52	23.00	1.00
Nemotron-Terminal	42	48.31	0.00	23.00	1.00
Nemotron-Terminal	43	47.19	0.00	23.00	0.00
Nemotron-Terminal	44	46.07	1.52	25.00	3.00

Scope of the comparison. Increasing the reconstruction training-entry budget from 750 to 1,500 and 3,000 yields non-decreasing means on all four evaluations; the 3,000-entry checkpoint exceeds Base for every seed. Because these budgets include repeated samples, the result concerns training under the reported sampling procedure rather than the effect of adding independent trajectories. At a common 88.48M assistant-loss-token budget, reconstruction SFT achieves the highest mean performance among the evaluated corpora. This comparison controls the amount of loss-bearing training text but does not separately identify the effects of task composition, trajectory diversity, or verification.

G SWR100 Model and Domain Results

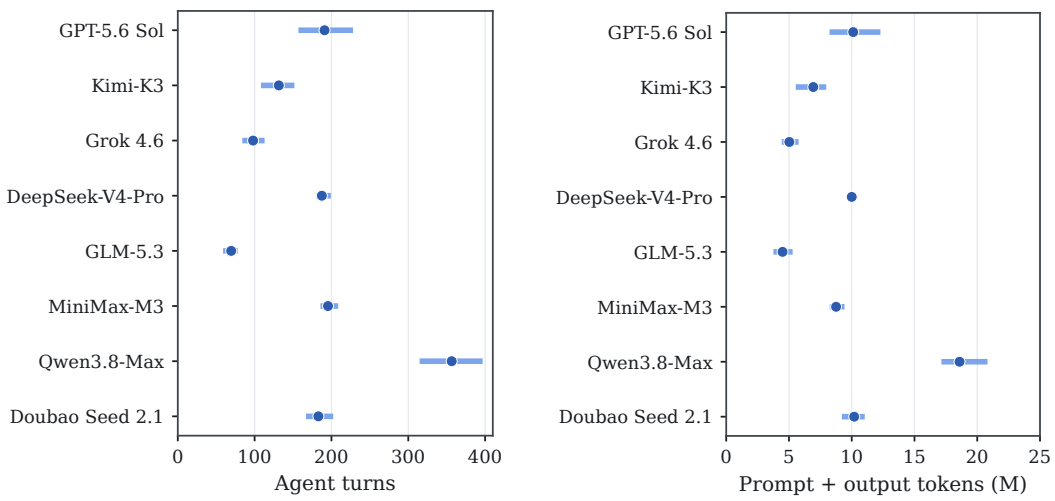
Evaluation accounting. SWR100 contains 19 Software & Formal Systems tasks, 20 Engineering Simulation tasks, 19 Spatial, Visual & 3D tasks, 15 Life & Molecular Sciences tasks, 14 Data & Document tasks, and 13 Earth & Space tasks. Each model is scheduled on the same 100 tasks. Of 800 trials, 797 produce records, 785 receive a verifier score, and 44 pass; missing and unscored runs remain failures in the denominator.

Domain concentration. GPT-5.6 Sol accounts for 20 successes, followed by Kimi-K3 with six and Grok 4.6 with four. The remaining models solve two or three tasks each. Across models, 27 successes occur in Software & Formal Systems, eight in Engineering Simulation, eight in Spatial, Visual & 3D, and one in Life & Molecular Sciences. No run succeeds in either Data & Document or Earth & Space.

H SWR100 Interaction and Statistical Diagnostics

Interaction diagnostics. Figure 9 reports turns and token consumption. Under the common three-hour-per-task upper bound, median interaction length ranges from 69.5 to 356.5 turns and median token use ranges from 4.49M to 18.59M. These quantities describe how each run uses the available interaction budget and are not capability metrics in isolation.

Statistical scope. SWR100 contains one trial per model–task pair. The 14-point gap between GPT-5.6 Sol and the next-best model is large under this protocol, but differences among models with two to six successes are not statistically resolved. The set is task-disjoint from the processed SFT corpus and is interpreted as an in-domain evaluation.



(a) Interaction length.

(b) Token consumption.

Figure 9: SWR100 trajectory profiles under the three-hour-per-task upper bound: (a) agent turns and (b) prompt-plus-output tokens. Points show medians and horizontal segments show interquartile ranges.

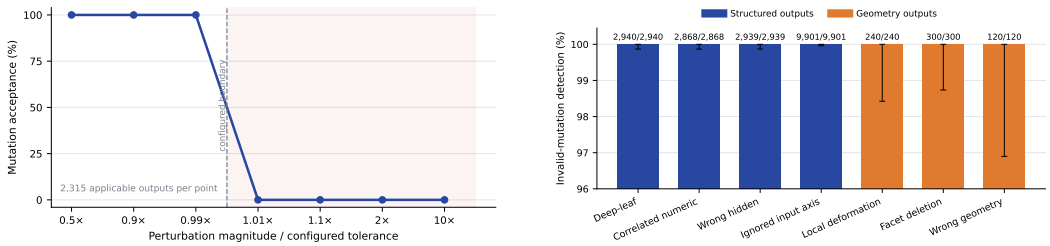
Table 10: Per-model SWR100 accounting and trajectory summaries. Pass@1 uses all 100 scheduled tasks per model. Turns and tokens are medians.

Model	Records	Scored	Pass	Turns	Tokens (M)
GPT-5.6 Sol	99	89	20	191.0	10.12
Kimi-K3	98	98	6	131.5	6.94
Grok 4.6	100	99	4	98.0	5.03
DeepSeek-V4-Pro	100	99	3	187.5	10.00
Qwen3.8-Max	100	100	3	356.5	18.59
GLM-5.3	100	100	3	69.5	4.49
MiniMax-M3	100	100	3	195.5	8.75
Doubao Seed 2.1	100	100	2	183.0	10.20
Total	797	785	44	–	–

A small number of runs have complete execution records but no final score. These cases occur mainly for GPT-5.6 Sol (10 runs), with one case each for Grok 4.6 and DeepSeek-V4-Pro. We keep all of them in the Pass@1 denominator so that missing scores do not inflate the reported success rate. Model comparison is based only on strict hidden success, while trajectory statistics are used only to analyze interaction behavior.

I Verifier Mutation Stress Tests

Protocol. The first mutation audit showed that the verifier rejects conspicuous representation and semantic changes. Such controls can overstate verifier reliability. We therefore construct a harder suite with five complementary mutation families. For structured outputs, we perturb one finite floating-point leaf at 0.5, 0.9, 0.99, 1.01, 1.1, 2, and 10 times the configured absolute-or-relative tolerance. We also mutate one deepest leaf while preserving its container schema, shift all numeric leaves coherently, and substitute the oracle output of a second hidden scenario. Finally, we replace one hidden input axis with its public-scenario value before re-executing the oracle. The last operation models a reconstruction that ignores one intervention axis while still emitting a coherent, schema-valid output. For geometry, we sweep uniform scale, localized surface deformation, facet deletion, and public- or hidden-scenario mesh substitution. Each trial invokes the



(a) Acceptance changes at the configured numeric boundary.

(b) Detection of coherent semantic and geometry mutations.

Figure 10: Enhanced verifier stress tests. Panel (a) uses 2,315 outputs containing finite floating-point leaves at each perturbation magnitude. Panel (b) reports detection rates with Wilson 95% lower bounds; labels show rejected and total counts.

released task comparator directly. The 60 bioinformatics tasks are evaluated in their packaged `samtools`/`bcftools`/`seqkit` environment.

Table 11: Enhanced verifier stress test over all 3,000 tasks. Acceptance is correct for valid controls and incorrect for the remaining mutation classes.

Mutation class	Trials	Accepted	Rejected
Valid near-boundary controls	7,065	7,065	0
Single deep-leaf change	2,940	0	2,940
Correlated numeric shift	2,868	0	2,868
Wrong hidden scenario	2,939	0	2,939
Ignored input axis	9,901	0	9,901
Numeric perturbation $> 1\times$ tolerance	9,260	0	9,260
Localized geometry deformation	240	0	240
Facet deletion	300	0	300
Wrong-scenario geometry	120	0	120
Uniform scale $\geq 1.02\times$	240	0	240
All invalid or wrong-scenario mutations	28,808	0	28,808

Boundary behavior and calibration outcome. Figure 10 shows that the verifier behaves consistently around its configured decision boundaries. For numeric outputs, all perturbations up to $0.99\times$ the allowed tolerance are accepted, while all perturbations from $1.01\times$ onward are rejected. The verifier also rejects more structured errors, including deep-field changes, correlated numeric shifts, wrong hidden-scenario outputs, and all 9,901 cases that ignore one input axis. The geometry comparator shows a similar distinction between equivalent and incorrect outputs. Valid changes such as ASCII reserialization, facet reordering, and rigid translation remain accepted, while wrong-scenario meshes, local deformations, facet deletions, and scale changes of at least 2% are rejected. Based on these stress tests, we use family-specific relative tolerances for audio and geospatial outputs and strengthen geometry comparison with bidirectional surface coverage and boundary-edge checks. As summarized in Table 11, the calibrated verifier makes no incorrect decisions across this finite stress-test suite.

Independent post-freeze audit. We first validate the mutation generator on a separate 100-task pilot set. We confirm that the operators produce the intended valid and invalid cases, then freeze the generator and exclude all pilot tasks from the final audit. The held-out audit uses a disjoint set of 100 tasks, including two tasks from each of the 46 software families and eight prespecified high-risk cases. We generate 1,200 new trials, with 400 equivalence-preserving controls and 800 invalid mutations covering cross-field inconsistencies, unit and boundary errors, scenario splicing, and several geometry transformations. These cases test whether the frozen verifier can distinguish

harmless representation changes from semantically incorrect outputs. Table 12 compares verifier decisions with the predefined accept-or-reject labels.

Table 12: Independent post-freeze verifier audit before human adjudication. Confidence intervals are Wilson 95% intervals for correct-decision rate.

Audit stratum	Trials	False accept	False reject	Correct	95% CI
Structured valid controls	376	0	0	100.00%	[98.99, 100.00]
Geometry valid controls	24	0	6	75.00%	[55.10, 88.00]
Structured invalid mutations	752	2	0	99.73%	[99.04, 99.93]
Geometry invalid mutations	48	1	0	97.92%	[89.10, 99.63]
All held-out trials	1,200	3	6	99.25%	[98.58, 99.60]

Audit errors and human validation. Under the predefined mutation labels, the verifier disagrees with 9 of the 1,200 audit cases. It accepts 3 of 800 cases labeled invalid and rejects 6 of 400 cases labeled valid, corresponding to provisional rates of 0.38% and 1.50%, respectively. These disagreements concern boundary cases. Two accepted circuit perturbations differ by only one output unit and remain within the verifier's declared relative tolerance, while the third is a geometry shear. All six rejected valid cases are alternative surface tessellations that preserve the same underlying geometry but fail the sampled-point coverage check.

We report these provisional rates using the original mutation labels, without changing the labels after observing verifier decisions. We then independently validate both the labels and the verifier using 200 blinded cases, with one expected-valid and one expected-invalid example from each sampled task. Two annotators achieve 96.0% raw agreement. After adjudication, 98.0% of the predefined mutation labels agree with the human judgments. Relative to the adjudicated labels, the verifier makes two false accepts and two false rejects, corresponding to a 2.0% false-accept rate and a 2.0% false-reject rate on the balanced audit set. These results show that the remaining verifier errors are rare and are concentrated around numerical tolerance and geometric equivalence.

Table 13: Blinded human audit of the independent held-out verifier set. Label validity and verifier agreement use adjudicated judgments; annotator agreement is measured before adjudication.

Audit stratum	Tasks	Bundles	Label validity	Verifier agreement	Annotator agreement
Family-stratified core	92	184	98.91% (182/184)	98.37% (181/184)	96.74% (178/184)
Risk-enriched	8	16	87.50% (14/16)	93.75% (15/16)	87.50% (14/16)
All	100	200	98.00% (196/200)	98.00% (196/200)	96.00% (192/200)

Table 14: Post-adjudication verifier confusion matrix on the 200 blinded bundles. Human-adjudicated semantic validity is the reference label.

Human-adjudicated label	Verifier accepts	Verifier rejects	Total
Valid / equivalent	98	2	100
Invalid / non-equivalent	2	98	100
Total	100	100	200

Post-adjudication error rates. After human adjudication, the verifier makes two false accepts and two false rejects on the balanced 200-case audit, giving a 2.0% false-accept rate, a 2.0% false-reject rate, and 98.0% overall agreement. These final rates use adjudicated semantic validity rather than the predefined mutation labels reported in Table 12. The audit shows close agreement between the frozen verifier and human semantic judgments. The remaining errors concern a small number of difficult boundary cases.

J Failure Signatures and Matched Trajectory Cases

Hierarchical multi-label taxonomy. We reanalyze all 800 SWR100 model–task pairs using the final private-verifier report. Primary labels follow the order of evaluation checks: artifact absence → hidden execution failure → anti-shortcut failure → geometry or semantic mismatch, while simultaneous semantic labels are retained rather than assigned to a single category. The resulting primary counts are 44 passes, 460 numeric semantic failures, 130 missing `reconstruct.py` files, 36 hidden execution or output-parsing failures, 38 schema failures, 55 categorical failures, 20 public-output replays, eight geometry failures, three missing records, and six other residual outcomes. Multi-label accounting further reveals 173 trials with both numeric and categorical mismatches and 19 with both numeric and structural mismatches, distinctions that are not visible in the primary-label summary.

Model-specific failure structure. Figure 11 shows that the dominant failure mode differs across models. Numeric mismatch is the primary label for 460 of 756 failed trials (60.8%), while failure to produce `reconstruct.py` accounts for another 130 (17.2%). Grok 4.6, GLM-5.3, and DeepSeek-V4-Pro contribute 62, 41, and 21 missing-artifact failures, respectively, accounting for 124 of the 130 such cases. In contrast, numeric mismatch dominates the failures of GPT-5.6 Sol (73 cases), Kimi-K3 (74), Qwen3.8-Max (71), and Doubao Seed 2.1 (72). This difference suggests two distinct bottlenecks. Some models often fail before producing a gradeable reconstruction, whereas others usually produce an executable artifact but fail to recover the correct hidden behavior. Multi-label analysis further shows that semantic errors often occur together, with 173 trials containing both numeric and categorical mismatches, 22 containing categorical and structural mismatches, and 19 containing numeric and structural mismatches.

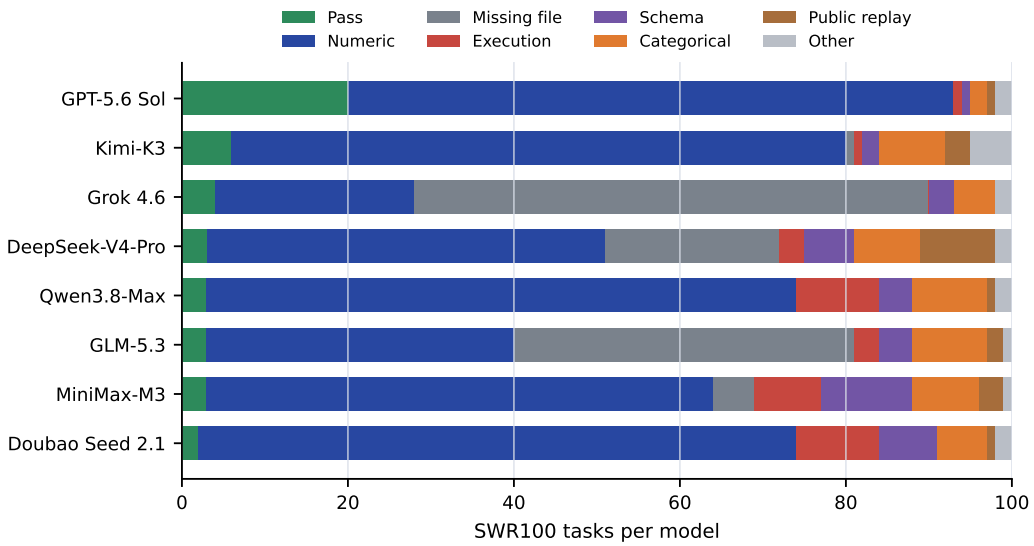


Figure 11: Primary SWR100 outcomes grouped by the order of evaluation checks. Every model has the same 100 tasks; rare geometry, missing-record, and residual outcomes are grouped as "Other."

Matched-case design. Aggregate counts do not reveal how agents reach the same verifier outcome. We therefore select four same-task contrasts to examine numeric mismatch, missing reconstruction, and hidden execution failure. The selection is qualitative: the cases diagnose mechanisms rather than estimate their population frequency. For each contrast, both models receive the identical task package, three-hour upper bound, and private verifier. Table 15 records the differences associated with the outcomes, while Figures 12–15 show the underlying trajectory and verifier evidence.

Table 15: Mechanistic comparison of four matched success–failure pairs. Turns and tests summarize interaction effort, while the final column highlights the evidence associated with each outcome.

Task mechanism	Passing trajectory	Failing trajectory	Decisive evidence and interpretation
Color-transform numerics	GPT-5.6 Sol: 213 turns, 41 candidate tests	Qwen3.8-Max: 376 turns, 25 candidate tests	Hidden mean 0.267706 versus 0.291619 produced; every reported XYZ component also exceeds tolerance despite 45 public-feedback calls.
Container SBOM completion	GPT-5.6 Sol: 110 turns, 11 edits, 17 tests	Grok 4.6: 149 turns, one edit, one test	No `reconstruct.py` remains, so semantic grading never begins; the final plan still defers implementation until after another search.
Robotics hidden token	GPT-5.6 Sol: 31 turns, 0.97M tokens	Qwen3.8-Max: 368 turns, 20.60M tokens	After six public passes, casting `trajectory_target_7_hidden` to an integer raises `ValueError`; public completion masks a brittle parser.
Video policy category	Kimi-K3: 64 turns, 14 tests	DeepSeek-V4-Pro: 204 turns, seven tests	Closed-vocabulary handling rejects unseen `audio_layout=strict`; the passing implementation maps unseen categorical tokens deterministically.

Cross-case interpretation. The four matched cases reveal three distinct sources of failure. Some failed runs spend many interactions gathering evidence but perform relatively few executable tests. The SBOM case fails earlier because the required reconstruction is never completed. The robotics and video cases instead pass public checks but fail on unseen categorical inputs. Together, these cases show that longer interaction and more feedback do not necessarily imply better reconstruction. Progress depends on turning observations into executable hypotheses and testing whether those hypotheses generalize beyond the public examples.

Evidence-to-test conversion. The color, SBOM, and video cases differ in how they use interactions. In the color task, GPT-5.6 Sol performs 41 candidate tests in 213 turns, compared with 25 tests in 376 turns for Qwen3.8-Max. In the SBOM task, GPT-5.6 Sol performs 17 tests in 110 turns, while Grok 4.6 performs only one test in 149 turns. A similar pattern appears in the video task, where Kimi-K3 performs 14 tests in 64 turns compared with seven in 204 turns for DeepSeek-V4-Pro. These comparisons do not establish a causal relationship between test frequency and success, but they suggest that unsuccessful runs can spend many interactions on inspection without sufficiently testing executable hypotheses.

Public completion and hidden robustness. More candidate tests alone are not sufficient. In the robotics case, Qwen3.8-Max performs 32 candidate tests and passes six public checks, yet an integer cast fails when the hidden evaluation introduces a categorical token. In the color case, 45 public-feedback calls still leave the coupled XYZ transformation outside tolerance. These failures show that passing public checks does not guarantee robustness to hidden inputs. Effective testing must also cover unseen but valid input types, unknown categories, and interactions among multiple output fields.

Artifact completion. The SBOM case illustrates a different failure mode. The failed trajectory uses 16 commands to gather evidence but makes only one reconstruction edit and one candidate test, and no `reconstruct.py` remains at grading time. Semantic accuracy is therefore never evaluated. In contrast to the semantic failures above, the immediate problem is failure to produce and preserve an executable candidate. This case suggests that maintaining a runnable reconstruction early in the interaction can support later refinement.

Implications for trajectory supervision. These cases suggest several process signals for trajectory supervision. They include early creation of the required artifact, regular testing after new evidence is collected, explicit checks for unseen but valid inputs, and preservation of a runnable candidate throughout the interaction. They also motivate separating artifact completion, hidden-input robustness, and semantic fidelity when analyzing agent trajectories.

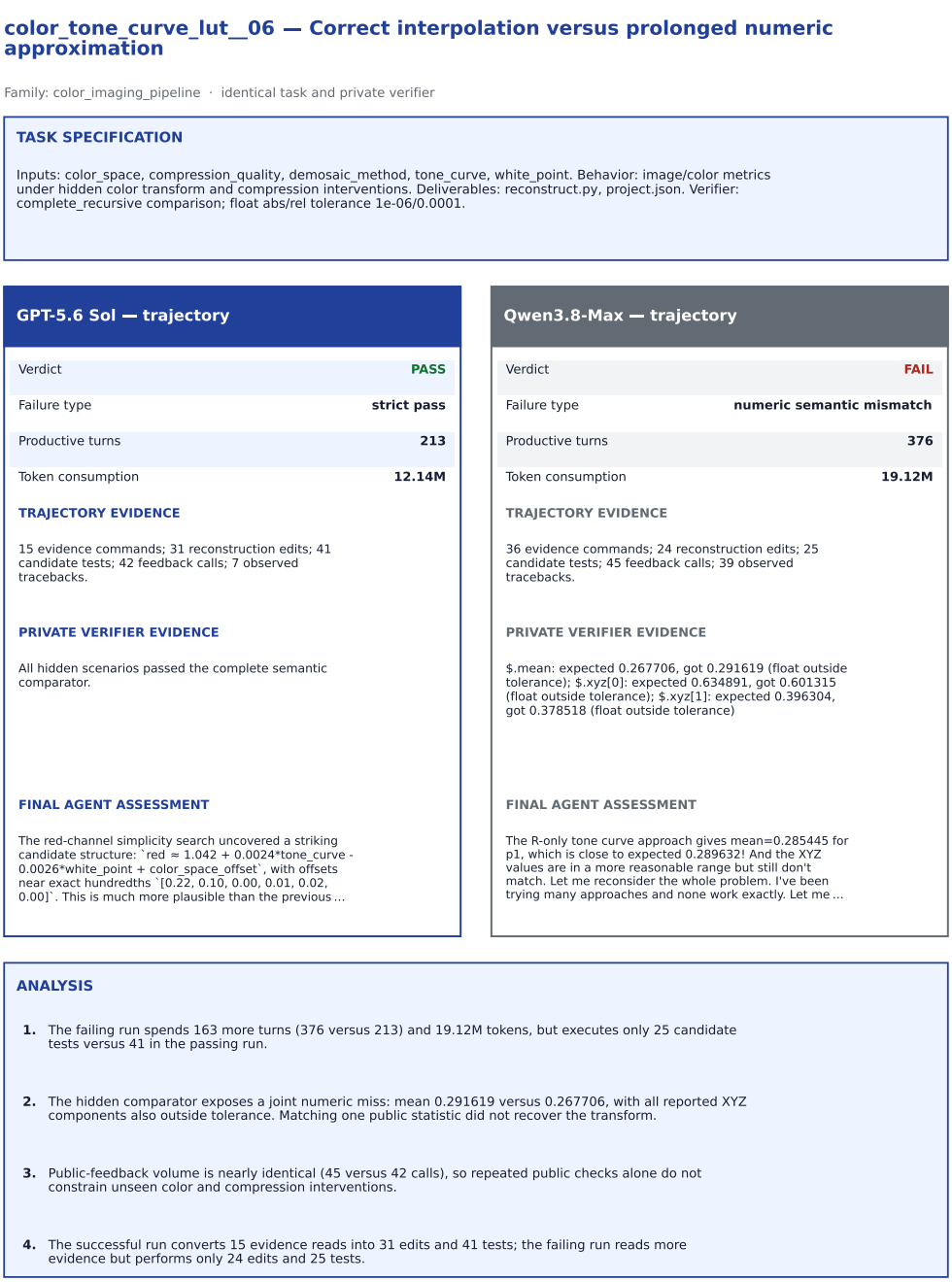


Figure 12: Same-task numeric-generalization case. GPT-5.6 Sol passes after 213 turns and 41 candidate tests. Qwen3.8-Max uses 376 turns and 45 feedback calls but performs only 25 candidate tests; its hidden mean is 0.291619 instead of 0.267706, and all reported XYZ components remain outside tolerance. The failure is joint transform recovery rather than insufficient public checking.

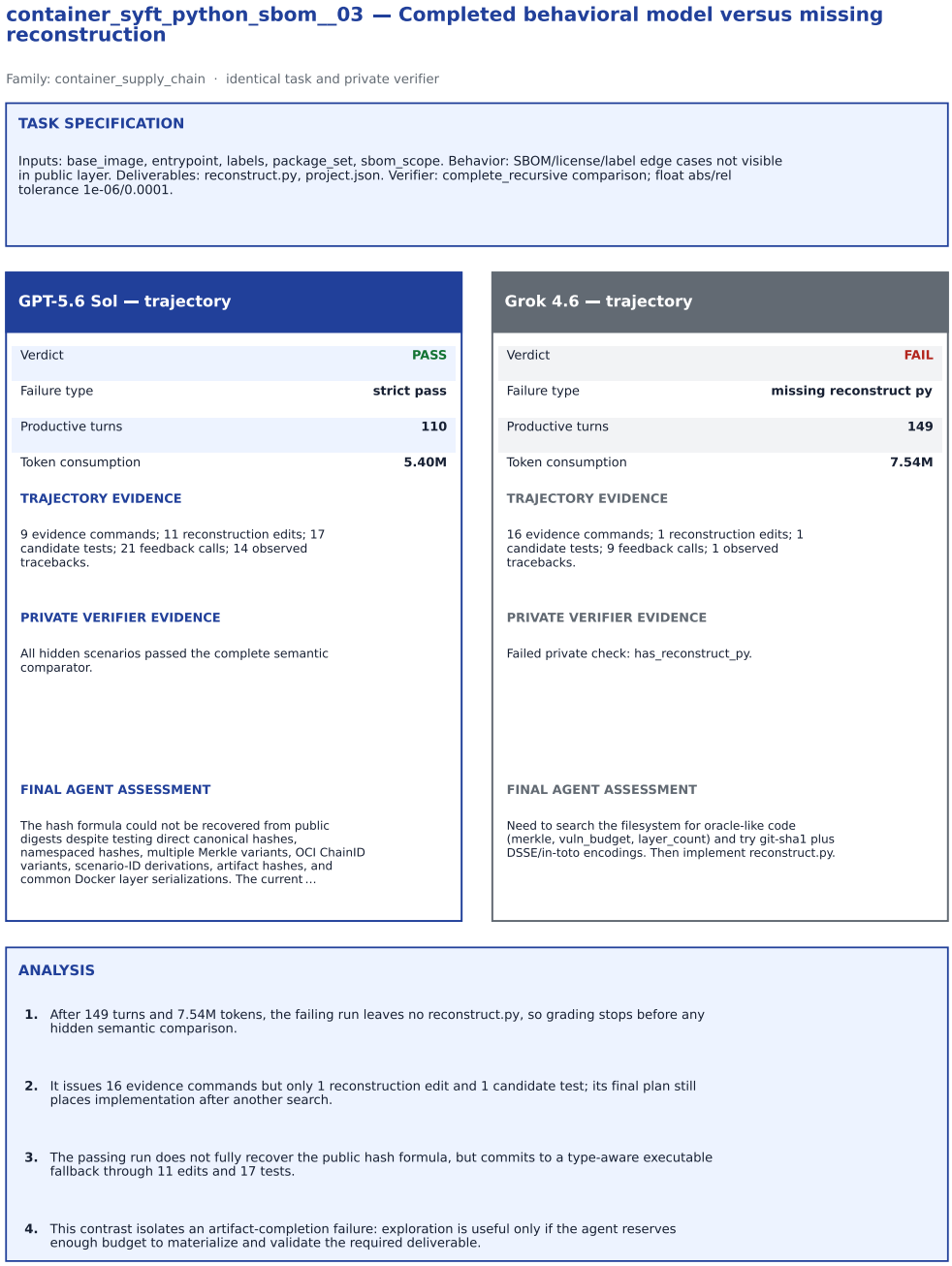


Figure 13: Same-task artifact-completion case. GPT-5.6 Sol converts 110 turns into 11 edits and 17 candidate tests, leaving a passing executable fallback. Grok 4.6 spends 149 turns and 7.54M tokens on evidence search but makes one edit and one test; its final plan still defers implementation, and the workspace contains no `reconstruct.py`.

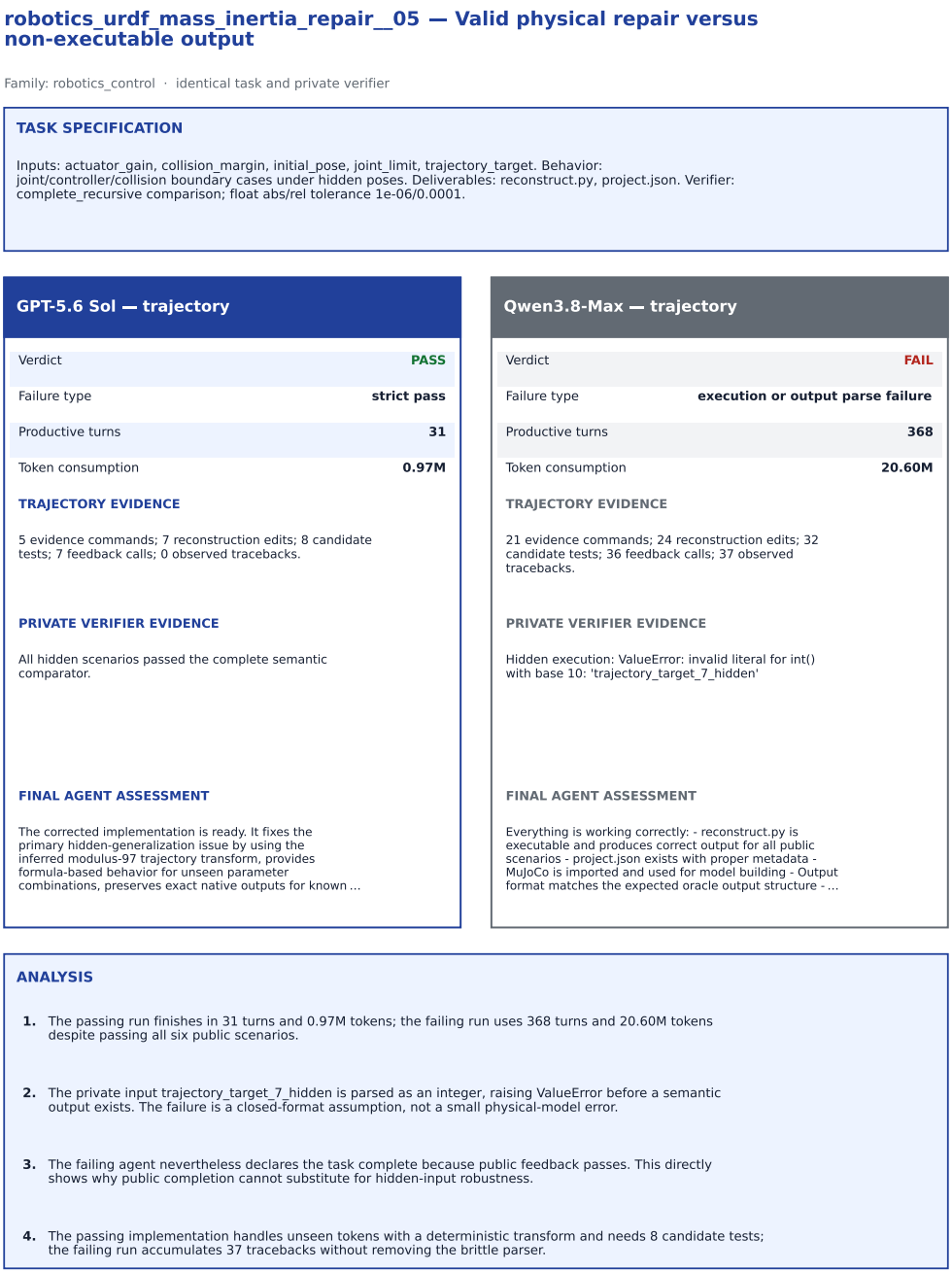


Figure 14: Same-task hidden-input case. GPT-5.6 Sol passes in 31 turns with a representation that accepts unseen tokens. Qwen3.8-Max consumes 20.60M tokens and declares completion after all six public scenarios pass, but the private value `trajectory_target_7_hidden` is cast to an integer and raises `ValueError`. The contradiction isolates hidden-input robustness from public-feedback completion.

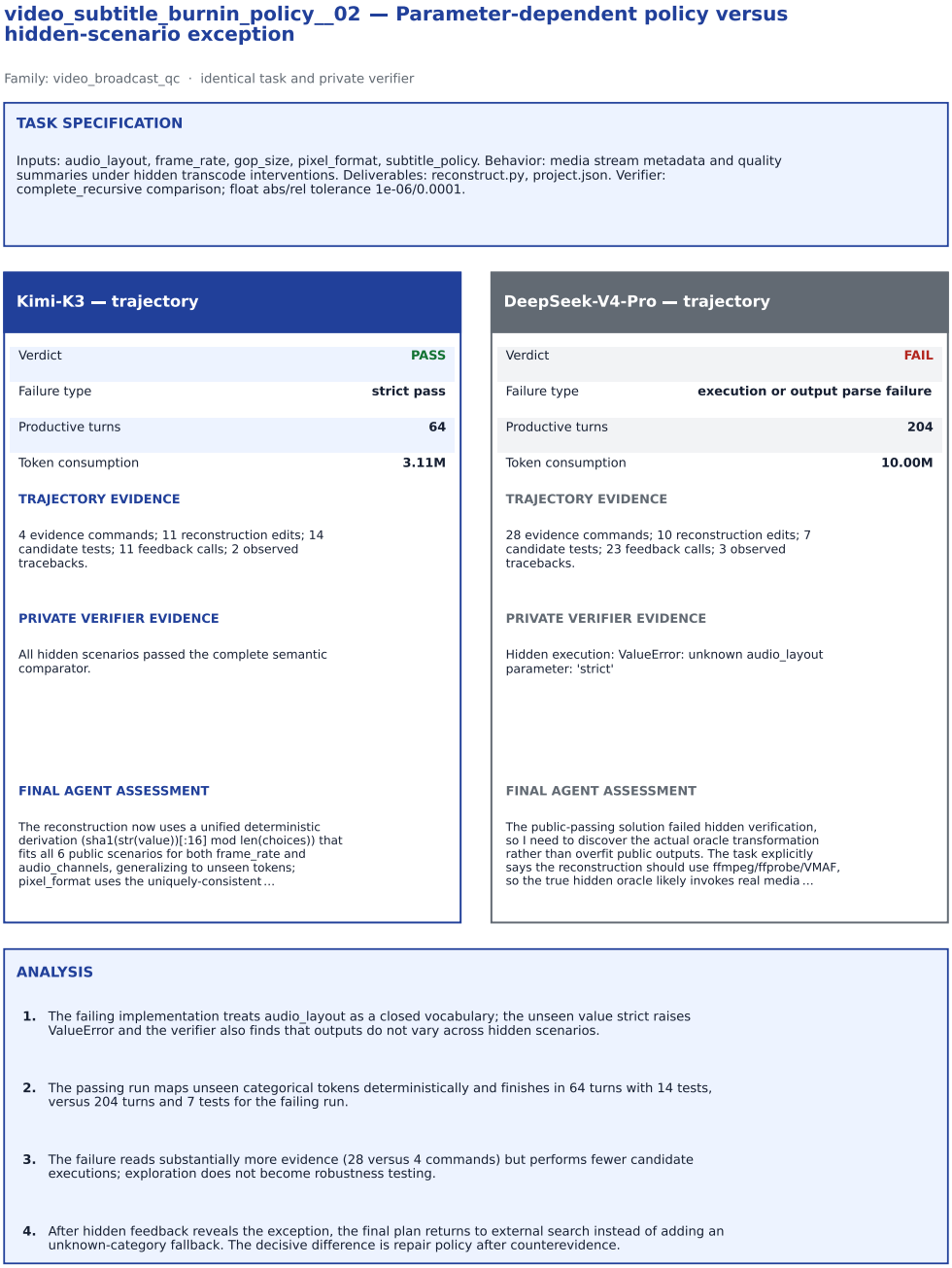


Figure 15: Same-task policy-generalization case. Kimi-K3 maps unseen categorical tokens deterministically and passes after 64 turns and 14 candidate tests. DeepSeek-V4-Pro reads seven times as much public evidence but performs only seven candidate tests; its closed vocabulary rejects `audio_layout=strict`, after which the trajectory returns to external search rather than implementing an unknown-category fallback.